

Project - High Level Design

on

Deploy Insurance App to Kubernetes

Course Name: DevOps Foundation

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	Sarthak Teredesai	EN22CS301879
02	Shrasti Gupta	EN22CS301925
03	Shruti Sharma	EN22CS301940
04	Saumya Kumari	EN22CS301886
05	Sanidhya Singh Chouhan	EN22CS301864

Group Name: 05D8

Project Number: DO-40

Industry Mentor Name: Ms. Aashruti Shah

University Mentor Name: Prof. Avnesh Kumar Joshi

Academic Year: 2025-26

Table of Contents

1. Introduction.
 - 1.1. Scope of the document.
 - 1.2. Intended Audience
 - 1.3. System overview.
2. System Design.
 - 2.1. Application Design
 - 2.2. Process Flow.
 - 2.3. Information Flow.
 - 2.4. Components Design
 - 2.5. Key Design Considerations
 - 2.6. API Catalogue.
3. Data Design.
 - 3.1. Data Model
 - 3.2. Data Access Mechanism
 - 3.3. Data Retention Policies
 - 3.4. Data Migration
4. Interfaces
5. State and Session Management
6. Caching
7. Non-Functional Requirements
 - 7.1. Security Aspects
 - 7.2. Performance Aspects
8. References

1. Introduction

The Multi-Agent Insurance System is a production-grade AI platform designed to intelligently process and respond to insurance policy queries using autonomous AI agents. Built on a modern cloud-native stack, the system leverages multiple CrewAI agents — Researcher, Explainer, and Comparator — orchestrated by a FastAPI backend, and deployed on a Minikube Kubernetes cluster using Docker containers.

This High Level Design (HLD) document describes the architectural blueprint, system components, data flows, interfaces, and non-functional considerations that guide the implementation of the system.

1.1. Scope Of the document

This document covers the high-level architecture and design of the Multi-Agent Insurance System. It is intended to provide a clear and comprehensive understanding of the following aspects:

- The overall system structure and its three-tier layered architecture (Frontend, Backend, AI Agents).
- Component-level design of the FastAPI backend, Streamlit frontend, and CrewAI agent framework.
- Kubernetes-based deployment strategy including Deployments, Services, ConfigMaps, and Secrets.
- Data flow between system layers, from user query to AI-generated response.
- API catalogue exposing all system endpoints with request/response specifications.
- Non-functional requirements including security, performance, and scalability.
- State management, caching strategy, and session handling.

This document does not cover low-level code implementation details, internal algorithm workings of the Gemini LLM, or infrastructure provisioning beyond the Minikube cluster scope.

1.2. Intended Audience

This document is intended for the following stakeholders:

Audience	Purpose / Relevance
Software Developers	To understand the architecture, component interactions, and API contracts for implementation and extension of the system.
DevOps / Cloud Engineers	To understand the Kubernetes deployment topology, rolling update strategy, and infrastructure configuration.
University & Industry Mentors	To evaluate the system design against project objectives and academic/industry standards.
QA / Testing Team	To derive test strategies based on system interfaces, API endpoints, and component boundaries.
Project Stakeholders	To review the overall design approach and validate alignment with business and functional requirements.

1.3. System Overview

The Multi-Agent Insurance System is a three-tier AI-powered web application that helps users research, understand, and compare insurance policies using natural language AI processing. The entire system is containerized with Docker and orchestrated on a Minikube Kubernetes cluster running on Windows.

The three core architectural layers are:

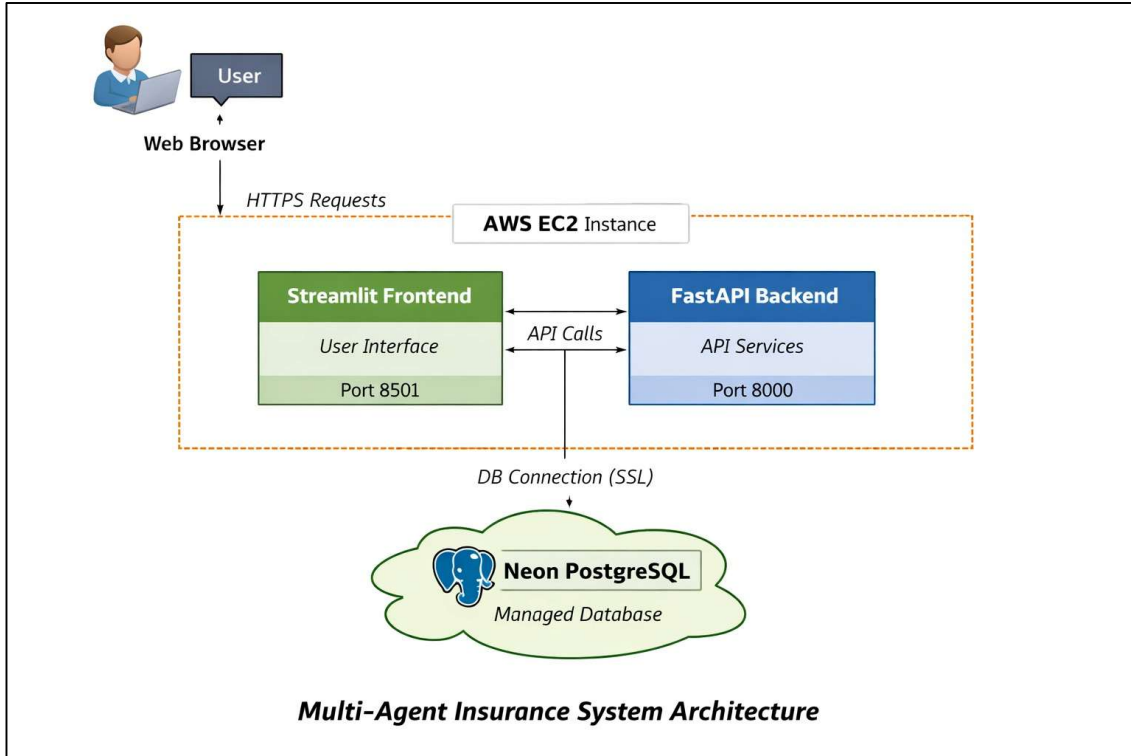
Layer	Technology	Responsibility
Presentation Layer	Streamlit (Python)	Provides a web-based UI for users to submit queries, browse policies, and view AI-generated explanations and comparisons.
Application Layer	FastAPI + CrewAI	Processes requests, orchestrates AI agents in sequential pipelines, validates inputs, and returns structured JSON responses.

Layer	Technology	Responsibility
AI / Intelligence Layer	CrewAI + Google Gemini 2.5 Flash	Autonomous agents (Researcher, Explainer, Comparator) call the Gemini LLM to generate policy insights and comparisons.

The platform runs on a zero-downtime Rolling Update strategy, ensuring continuous availability during version upgrades. Key platform attributes are summarised below:

Attribute	Details
Platform	Minikube (local Kubernetes on Windows)
Backend	FastAPI + CrewAI 1.9.3 + Google Gemini 2.5 Flash
Frontend	Streamlit (Python)
Container Runtime	Docker Desktop (Windows)
Orchestration	Kubernetes — Deployment, Service, ConfigMap, Secret
Update Strategy	RollingUpdate — maxSurge: 1, maxUnavailable: 0
Agent Framework	CrewAI with google-genai native provider
LLM	gemini/gemini-2.5-flash via GOOGLE_API_KEY

Sensitive configuration such as the Google API key is injected at runtime via Kubernetes Secrets and never hardcoded in any source file. The system is designed to be stateless at the agent level, with each request spawning fresh agent instances that interact with the Gemini LLM and return structured results.



2. System Design

This section describes the architecture, component structure, process flows, information flows, and API design of the Multi-Agent Insurance System. The system is designed as a layered, containerised, and cloud-native application deployed on Kubernetes.

2.1. Application Design

The system follows a three-tier application architecture. Each tier is independently containerised and communicates over defined interfaces.

Tier	Technology	Description
Presentation Tier	Streamlit (Python)	Web-based frontend served at port 8501. Renders policy dropdowns, research results, and comparison outputs. Communicates with the backend via HTTP REST calls.

Tier	Technology	Description
Application Tier	FastAPI (Python)	REST API backend served at port 8000. Exposes 4 endpoints, validates requests using Pydantic, orchestrates CrewAI agent pipelines, and returns structured JSON.
AI / Agent Tier	CrewAI + Gemini	Three stateless agents (Researcher, Explainer, Comparator) instantiated per request. Each agent calls Gemini 2.5 Flash over HTTPS and returns structured output to the backend.

Kubernetes Deployment Topology:

Resource	Type	Replicas / Count	Port Mapping
insurance-api	Deployment	2 replicas	8000 → NodePort 30080
insurance-frontend	Deployment	1 replica	8501 → NodePort 30081
insurance-api-service	ClusterIP + NodePort	—	80 → 8000 → 30080
insurance-frontend-service	ClusterIP + NodePort	—	80 → 8501 → 30081
insurance-app-config	ConfigMap	—	Env vars injected at runtime
insurance-app-secrets	Secret	—	GOOGLE_API_KEY (base64)

2.2. Process Flow

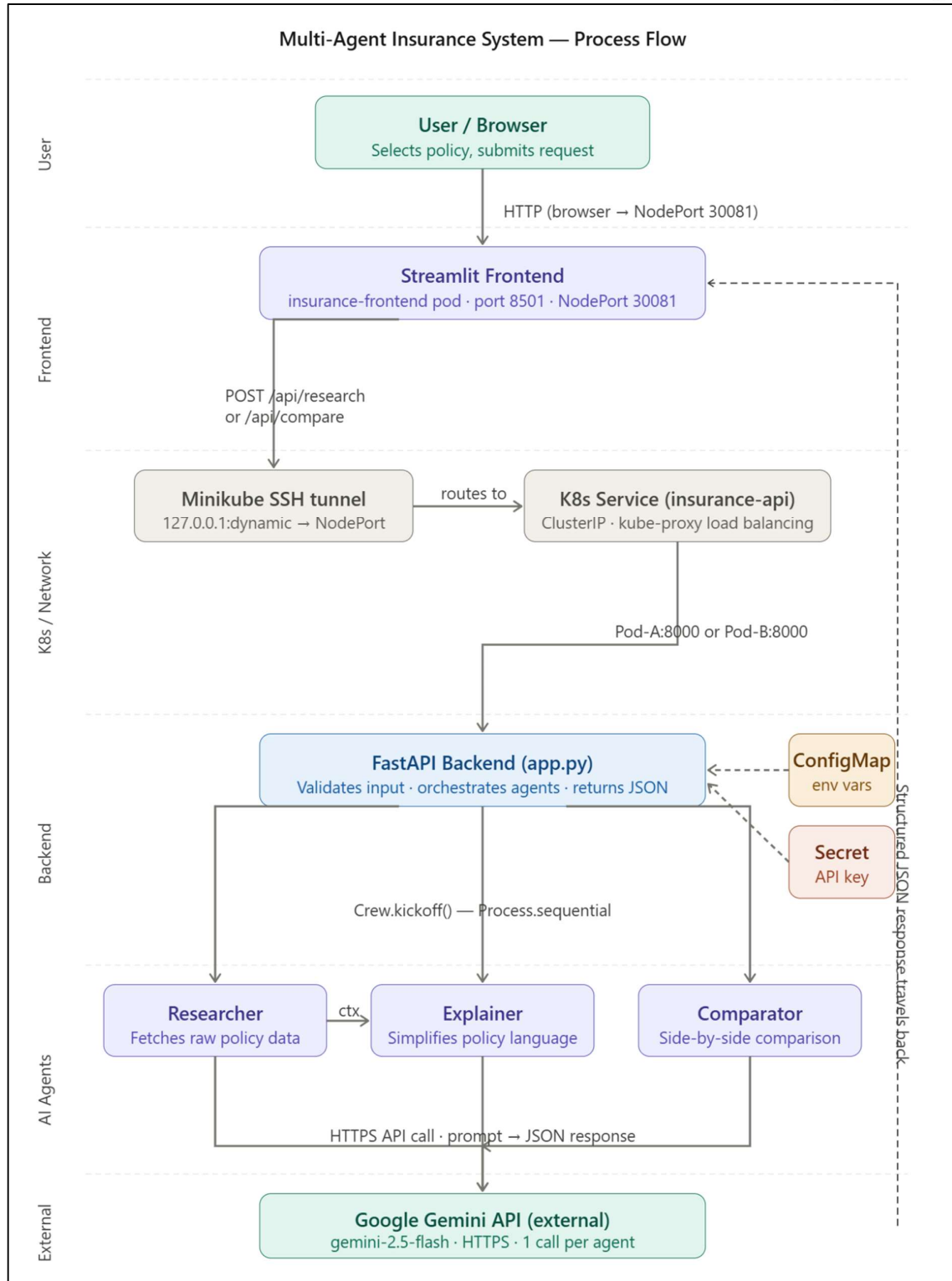
The system supports three primary user workflows: Health Check, Policy Research, and Policy Comparison. Each workflow follows a strict sequential request-response chain through the architectural layers.

Research Policy — Process Flow:

Step	Actor / Component	Action
1	User / Browser	Selects a policy from the dropdown and clicks 'Research Policy' in the Streamlit UI.
2	Streamlit Frontend	Sends POST /api/research with payload {policy_name: str} to the backend via Kubernetes internal DNS.
3	Minikube SSH Tunnel	Routes request from 127.0.0.1:dynamic-port → NodePort 30080 on the Minikube VM.
4	Kubernetes Service	kube-proxy load-balances the request to one of the two backend pods (Pod-A:8000 or Pod-B:8000).
5	FastAPI (app.py)	Validates policy name via validate_policy(). Instantiates Researcher and Explainer agents with Gemini LLM.
6	CrewAI — Researcher	Executes research_task: calls Gemini API over HTTPS, retrieves raw policy data as structured JSON.
7	CrewAI — Explainer	Executes explanation_task using Researcher output: generates simple_explanation, who_should_buy, who_should_avoid, key_takeaways.
8	FastAPI (app.py)	Calls clean_llm_json() to strip markdown fences, parses JSON, and returns structured HTTP response.
9	Streamlit Frontend	Renders the response: displays explanation, recommendations, and key takeaways to the user.

Compare Policies — Process Flow:

Step	Actor / Component	Action
1	User / Browser	Selects two policies (Policy A and Policy B) and clicks 'Compare'.
2	Streamlit Frontend	Sends POST /api/compare with payload {policy_a: str, policy_b: str}.
3	FastAPI (app.py)	Validates both policies. Runs Phase 1: Researcher agent fetches data for both policies in parallel tasks.
4	CrewAI — Phase 1	Crew([researcher], [task_a, task_b]).kickoff() — Researcher calls Gemini twice and returns raw data for both policies.
5	CrewAI — Phase 2	Crew([comparator], [comparison_task]).kickoff() — Comparator agent calls Gemini to produce side-by-side comparison and recommendation.
6	FastAPI (app.py)	Cleans and parses LLM JSON output. Returns structured comparison response.
7	Streamlit Frontend	Renders recommendation and comparison{} sections in the UI.



2.3. Information Flow

Information flows unidirectionally through the system layers in a strict request-response pattern. The diagram below describes the data exchanged at each communication boundary.

From	To	Protocol / Medium	Data Exchanged
User Browser	Streamlit Frontend	HTTP (NodePort 30081)	User interactions: policy selection, button clicks
Streamlit Frontend	FastAPI Backend	HTTP REST (Kubernetes DNS)	JSON payload: {policy_name} or {policy_a, policy_b}
FastAPI Backend	CrewAI Agents	Python function calls (in-process)	Policy name string, task definitions
CrewAI Agents	Google Gemini API	HTTPS REST (external)	Prompt text; receives JSON-structured LLM response
Google Gemini API	CrewAI Agents	HTTPS REST (external)	LLM output: raw policy facts, explanations, comparisons
CrewAI Agents	FastAPI Backend	Python return values (in-process)	Structured JSON: explanation, comparison, recommendation
FastAPI Backend	Streamlit Frontend	HTTP REST response	Cleaned JSON: key_takeaways, who_should_buy, comparison{}
Streamlit Frontend	User Browser	HTTP (rendered HTML)	Formatted UI: policy cards, comparison tables, recommendations
Kubernetes Secrets	FastAPI Backend Pod	Env var injection (kubelet)	GOOGLE_API_KEY at container startup

From	To	Protocol / Medium	Data Exchanged
Kubernetes ConfigMap	Both Pods	Env var injection (kubelet)	APP_ENV, CREWAI_DISABLE_TELEMETRY, LANGCHAIN_TRACING_V2

2.4. Components Design

The system is composed of six primary components. Each component has a clearly defined responsibility and communicates with others through well-defined interfaces.

Component	File / Resource	Responsibility	Interfaces
Streamlit Frontend	streamlit_app.py	Renders the web UI. Manages three pages: Health Check, Research Policy, Compare Policies. Reads BACKEND_URL from env to locate the API.	HTTP REST to FastAPI backend via BACKEND_URL env var
FastAPI Backend	backend/app.py	Entry point for all API requests. Validates input with Pydantic, orchestrates agent crews, and serialises JSON responses.	Exposes 4 REST endpoints. Calls CrewAI internally.
Researcher Agent	agents/researcher.py	Stateless agent that fetches raw insurance policy data. Instantiated per request with a Gemini LLM instance.	Python call from app.py; calls Gemini API via CrewAI

Component	File / Resource	Responsibility	Interfaces
Explainer Agent	agents/explainer.py	Stateless agent that takes Researcher output and produces simplified explanations: who_should_buy, who_should_avoid, key_takeaways.	Receives context from Researcher task output
Comparator Agent	agents/comparator.py	Stateless agent that performs side-by-side policy comparison and generates a structured recommendation.	Receives dual-policy context from Phase 1 crew
LLM Factory	config/settings.py	Single get_llm() factory function returns a configured Gemini 2.5 Flash LLM instance with temperature=0.3. API key injected from env.	Called by all three agents on instantiation
Policy Validator	utils/policy_validator.py	validate_policy() and get_all_policies() functions. Validates policy names against a known list before dispatching to agents.	Called by app.py before any agent crew kickoff
Kubernetes Manifests	k8s/*.yaml	Defines Deployments, Services, ConfigMap, and Secret. Controls replica count, rolling update strategy, health probes, and resource limits.	Applied via kubectl; manages all pod lifecycle

2.5. Key Designs Considerations

The following design decisions were made to ensure the system is robust, maintainable, secure, and operationally stable:

Design Consideration	Decision Made	Rationale
Agent Statelessness	Agents are instantiated fresh on every request with no shared state.	Ensures horizontal scalability. Multiple pods can handle concurrent requests independently without conflict.
Zero-Downtime Deployments	RollingUpdate strategy with maxSurge:1 and maxUnavailable:0.	New pods must pass readinessProbe before old pods are removed, guaranteeing no traffic interruption during updates.
Secret Management	GOOGLE_API_KEY stored in Kubernetes Secret; injected via envFrom.secretRef.	Prevents API key exposure in source code or Docker images. Key is base64-encoded at rest in etcd.
Internal DNS Routing	Frontend uses http://insurance-api-service/api via BACKEND_URL env var.	Eliminates hardcoded IPs that break between Minikube sessions. Kubernetes DNS resolves the service name to ClusterIP automatically.
LLM Temperature Control	temperature=0.3 in get_llm() factory.	Low temperature reduces hallucinations and variability in structured insurance data output, ensuring consistent JSON responses.
Request Timeouts	Research timeout=120s; Compare timeout=180s in Streamlit.	CrewAI agent chains take 30–120s due to sequential LLM calls. Without explicit

Design Consideration	Decision Made	Rationale
		timeouts, requests would hang indefinitely.
JSON Output Cleaning	clean_llm_json() strips markdown code fences before JSON parsing.	Gemini sometimes wraps JSON in markdown backtick blocks. This utility ensures safe parsing regardless of LLM formatting habits.
Resource Limits	CPU: 250m request / 500m limit. Memory: 256Mi request / 512Mi limit.	Prevents any single pod from consuming excessive cluster resources while reserving sufficient capacity for LLM inference calls.

2.6. API Catalogue

The FastAPI backend exposes four REST endpoints. All endpoints are prefixed with /api and served at port 8000 inside the container (accessible externally via NodePort 30080 through the Minikube tunnel).

Method	Endpoint	Agent(s) Used	Timeout	Purpose
GET	/api/system/health	None	10s	Health check
GET	/api/policies	None	—	List all policies
POST	/api/research	Researcher, Explainer	120s	Research a policy
POST	/api/compare	Researcher, Comparator	180s	Compare two policies

3. Data Design

This section describes how data is structured, accessed, retained, and migrated within the Multi-Agent Insurance System. As the system is primarily a stateless AI inference

platform, persistent storage is minimal. Data primarily flows through in-memory structures, environment-injected configuration, and Kubernetes-managed secrets.

3.1. Data Model

The system does not use a traditional relational database. All data is transient and lives within the request-response lifecycle. The primary data entities are defined below:

Entity	Type	Source / Owner	Description
PolicyName	String	policy_validator.py	A validated string identifier for an insurance policy. Used as input to all agent pipelines.
PolicyList	List[str]	policy_validator.py	A static list of all available policy names. Returned by GET /api/policies. No DB query involved.
ResearchResult	JSON Object	Researcher Agent (LLM)	Raw policy data returned by the Researcher agent after Gemini LLM inference. Contains unstructured policy facts.
ExplainedPolicy	JSON Object	Explainer Agent (LLM)	Structured output with fields: simple_explanation, who_should_buy, who_should_avoid, key_takeaways.
ComparisonResult	JSON Object	Comparator Agent (LLM)	Structured output with fields: recommendation (string), comparison{} (nested key-value breakdown of both policies).
HealthStatus	JSON Object	FastAPI (hardcoded)	Static JSON with keys: researcher_agent, explainer_agent, comparator_agent, llm_status — all set to 'active'.
GOOGLE_API_KEY	String (Secret)	Kubernetes Secret	Sensitive LLM API key. Base64-encoded in etcd. Injected as

Entity	Type	Source / Owner	Description
			environment variable into backend containers at pod startup.
AppConfig	Env Variables	Kubernetes ConfigMap	Non-sensitive runtime configuration: APP_ENV, CREWAI_DISABLE_TELEMETRY, LANGCHAIN_TRACING_V2.

Request/Response Data Schema:

Endpoint	Direction	Schema
POST /api/research	Request	{ "policy_name": "string" }
POST /api/research	Response	{ "simple_explanation": "str", "who_should_buy": "str", "who_should_avoid": "str", "key_takeaways": ["str"] }
POST /api/compare	Request	{ "policy_a": "string", "policy_b": "string" }
POST /api/compare	Response	{ "recommendation": "str", "comparison": { "field": { "policy_a": "str", "policy_b": "str" } } }
GET /api/policies	Response	["policy_name_1", "policy_name_2", ...]
GET /api/system/health	Response	{ "researcher_agent": "active", "explainer_agent": "active", "comparator_agent": "active", "llm_status": "active" }

3.2. Data Access Mechanism

The system employs multiple data access patterns depending on the data type:

Data Type	Access Mechanism	Details
Policy Names	In-memory / Static	Loaded at module import from policy_validator.py. get_all_policies() returns a Python list; validate_policy() checks membership. No I/O operation.

Data Type	Access Mechanism	Details
LLM API Key	Environment Variable	Injected by Kubernetes via envFrom.secretRef. Accessed in Python via <code>os.getenv('GOOGLE_API_KEY')</code> . Never read from disk or hardcoded.
App Configuration	Environment Variable	Injected via envFrom.configMapRef. APP_ENV, CREWAI_DISABLE_TELEMETRY, LANGCHAIN_TRACING_V2 read via <code>os.getenv()</code> in <code>config/settings.py</code> .
LLM Responses	HTTPS REST API	CrewAI agents call Google Gemini API over HTTPS. Responses are received as LiteLLM response objects, converted to string, then parsed via <code>clean_llm_json()</code> .
Backend URL (Frontend)	Environment Variable	Streamlit reads BACKEND_URL via <code>os.environ.get()</code> . Defaults to <code>http://insurance-api-service/api</code> . Resolved via Kubernetes cluster DNS.
Request/Response Data	HTTP REST + Pydantic	FastAPI uses Pydantic models to validate and deserialise incoming JSON. Responses are serialised automatically by FastAPI's <code>JSONResponse</code> .

3.3. Data Retention Policies

Since the system is stateless and does not use a persistent database, data retention is minimal and well-defined:

Data Category	Storage Location	Retention Duration	Notes
LLM Request/Response	In-memory (RAM)	Duration of HTTP request only	No logging of LLM inputs or outputs. Data is discarded once the HTTP response is sent.
Agent Task Results	In-memory (RAM)	Duration of request lifecycle	Passed between agents as Python objects within a single <code>Crew.kickoff()</code> call. Not persisted.
GOOGLE_API_KEY	Kubernetes etcd (base64)	Until Secret is deleted or rotated	Persisted in cluster state. Should be rotated

Data Category	Storage Location	Retention Duration	Notes
			periodically. Never written to logs or stdout.
ConfigMap Values	Kubernetes etcd	Until ConfigMap is updated/deleted	Non-sensitive. APP_ENV and telemetry flags persist across pod restarts.
Container Logs	Pod stdout (kubelet)	Until pod is terminated or log rotation	FastAPI/uvicorn and Streamlit logs are available via kubectl logs. Not persisted to disk by default.
Policy Name List	Python module memory	Lifetime of the container process	Loaded once on import. Refreshed only on pod restart or redeployment.

Future Recommendation: A PersistentVolumeClaim (PVC) should be mounted to store agent execution logs across pod restarts for audit and debugging purposes.

3.4. Data Migration

As the system currently has no persistent database, formal data migration procedures are not applicable. However, the following migration considerations apply for future evolution of the system:

Migration Scenario	Approach
Adding new insurance policies	Update the policy list in utils/policy_validator.py. Rebuild the Docker image and perform a Rolling Update via rolling-update.ps1. No DB migration needed.
Rotating GOOGLE_API_KEY	Update the Kubernetes Secret using kubectl create secret generic with --dry-run=client kubectl apply -f -. All running pods automatically receive the updated key on next restart.
Introducing a database (future)	Deploy a PostgreSQL or Redis StatefulSet in the cluster. Migrate policy data and agent result caching to DB. Use Kubernetes PersistentVolumes for storage.

Migration Scenario	Approach
Moving to cloud Kubernetes (future)	Migrate manifests from Minikube to AKS/EKS/GKE. Secrets should be migrated to cloud-native secret managers (AWS Secrets Manager, Azure Key Vault). No application code changes required.
Upgrading Gemini LLM model	Update the model string in config/settings.py. Rebuild and redeploy via Rolling Update. Validate agent output schemas as new models may return differently structured JSON.

4. Interfaces

The Multi-Agent Insurance System exposes and consumes several interfaces across its layers. These include user-facing interfaces, inter-service REST interfaces, external API interfaces, and Kubernetes-level configuration interfaces.

4.1 User Interface (UI)

The Streamlit frontend provides the primary user interface, accessible via a web browser at the Minikube tunnel URL. It consists of three functional pages:

UI Page	User Actions	Backend Interaction
Research Policy	Select policy from dropdown; click 'Research'	GET /api/policies (populate dropdown); POST /api/research (fetch AI explanation)
Compare Policies	Select two policies from dropdowns; click 'Compare'	GET /api/policies (populate both dropdowns); POST /api/compare (fetch AI comparison)

4.2 Internal Service Interface (Frontend ↔ Backend)

Protocol	HTTP REST (JSON)
Base URL	http://insurance-api-service/api (Kubernetes internal DNS)
Resolved By	Kubernetes CoreDNS — resolves 'insurance-api-service' to backend ClusterIP
External Access	Minikube SSH tunnel: 127.0.0.1:dynamic-port → NodePort 30080

Auth / Security	No authentication on internal service interface (trusted cluster network)
Content Type	application/json for all request and response bodies
Error Handling	Streamlit wraps all calls in try/except: ConnectionError, Timeout, JSONDecodeError handled gracefully

4.3 External API Interface (Backend ↔ Google Gemini)

Protocol	HTTPS REST (external)
Provider	Google Generative AI — gemini/gemini-2.5-flash
Called By	CrewAI agents (Researcher, Explainer, Comparator) via LiteLLM abstraction layer
Authentication	GOOGLE_API_KEY passed as API key header — injected from Kubernetes Secret
Request Format	Prompt text constructed by CrewAI task definitions; sent as LLM chat messages
Response Format	LLM text response — may include markdown code fences; cleaned by clean_llm_json()
Failure Handling	LLM failures propagate as 500 Internal Server Error to the Streamlit client

4.4 Kubernetes Configuration Interface

Resource	Interface Type	Consumed By
insurance-app-config (ConfigMap)	Environment Variables	Both backend and frontend pods via envFrom.configMapRef
insurance-app-secrets (Secret)	Environment Variables	Backend pods only via envFrom.secretRef — injects GOOGLE_API_KEY
insurance-api-service (Service)	ClusterIP + NodePort	Frontend pod (internal DNS); Windows host (NodePort 30080 via tunnel)

Resource	Interface Type	Consumed By
insurance-frontend-service (Service)	ClusterIP + NodePort	Windows host browser (NodePort 30081 via Minikube tunnel)
readinessProbe / livenessProbe	HTTP GET /api/system/health	Kubernetes kubelet — controls pod traffic eligibility and restart policy

5. State and Session Management

The Multi-Agent Insurance System is designed as a stateless application at both the backend service level and the AI agent level. This is a deliberate architectural decision to ensure horizontal scalability and simplicity of deployment.

5.1 Backend Statelessness

Aspect	Details
Agent Lifecycle	Agents (Researcher, Explainer, Comparator) are instantiated fresh on every HTTP request. They hold no state between requests. Once the Crew.kickoff() completes, all agent objects are garbage-collected.
Request Isolation	Each incoming HTTP request to /api/research or /api/compare spawns its own independent agent pipeline. There is no shared mutable state between concurrent requests.
No Server-Side Session	FastAPI does not maintain HTTP sessions. There are no session tokens, cookies, or user identity tracking at the backend level.
LLM Context	The Gemini LLM is called statelessly per agent task. No conversation history is maintained between requests. Each agent call is a fresh prompt-response cycle.
Pod Replicas	With 2 backend pod replicas, Kubernetes load-balances requests between pods. Because the backend is stateless, any pod can handle any request without session affinity.

5.2 Frontend Session Behaviour (Streamlit)

Aspect	Details
Streamlit Session State	Streamlit maintains in-browser session state per user tab using its built-in <code>st.session_state</code> . This is client-scoped and ephemeral — it is lost on page refresh.
Policy Dropdown State	The selected policy name is held in Streamlit widget state during the active session. It is not persisted to any backend store.
Research / Comparison Results	Results returned from the API are rendered directly to the UI and not stored. Navigating away from the page clears the results.
No Cross-User State	Each browser tab runs an independent Streamlit session. There is no shared state or user data crossing between sessions.
Timeout Handling	Requests that exceed timeout thresholds (120s/180s) are caught with a Timeout exception and displayed as user-friendly error messages. No partial state is retained.

5.3 State Summary

Layer	Stateful?	State Location	Persistence
FastAPI Backend	No	None	No state persisted
CrewAI Agents	No	In-memory (per request)	Discarded after response
Google Gemini API	No	External (stateless API)	No history retained
Streamlit Frontend	Partial	Browser session (<code>st.session_state</code>)	Lost on page refresh
Kubernetes ConfigMap	Yes	etcd (cluster state)	Persisted until updated
Kubernetes Secret	Yes	etcd (cluster state)	Persisted until rotated

6. Caching

The current version of the Multi-Agent Insurance System does not implement application-level caching. This is a deliberate design choice given the dynamic, LLM-generated nature of responses and the current local deployment scope. However, caching mechanisms exist at infrastructure and container levels.

Current Caching State

Cache Layer	Status	Details
Application-level cache (LLM responses)	Not Implemented	Each /api/research and /api/compare request triggers a fresh LLM call. Identical queries for the same policy will invoke Gemini each time.
Policy list cache	Implicit (module-level)	get_all_policies() returns a static Python list loaded at module import. This is effectively cached in memory for the lifetime of the pod process.
Docker layer cache	Active (build-time)	Docker caches each build layer. The pip install layer is reused unless requirements.txt changes. Use --no-cache flag to force a full rebuild.
Kubernetes node cache	Active (image pull)	imagePullPolicy: Never means Kubernetes always uses the locally available image inside the Minikube daemon. No network pull on pod start.
Browser cache	Streamlit default	Streamlit does not implement explicit browser caching of API responses. UI state is reset on page reload.

7. Non-Functional Requirements

This section defines the non-functional requirements (NFRs) of the Multi-Agent Insurance System, covering security, performance, scalability, reliability, and maintainability.

7.1 Security Aspects

Security is addressed at multiple layers of the system — from secret management and container isolation to Kubernetes network boundaries.

Security Concern	Current Implementation	Status	Recommendation
API Key Management	GOOGLE_API_KEY stored in Kubernetes Secret; base64-encoded in etcd; injected via envFrom.secretRef	Implemented	Enable etcd encryption at rest for full encryption of Secret values (not just base64 encoding).
Secret Exposure in Code	API key never hardcoded in source files, Dockerfiles, or committed .env files	Implemented	Add .gitignore rules for .env and enforce via pre-commit hooks.
Container Root User	Containers run as root user by default (standard Python base image)	Partial	Add USER directive in Dockerfile to run uvicorn as a non-root user inside the container.
Pod-to-Pod Network	All pods in default namespace; frontend can reach backend and external internet	Partial	Implement Kubernetes NetworkPolicies to restrict frontend-to-backend-only traffic and block direct external access to backend pods.
HTTPS (External)	Gemini API calls use HTTPS; frontend-to-backend traffic uses HTTP inside the cluster	Partial	Deploy NGINX Ingress Controller with TLS termination to enforce HTTPS for all user-facing traffic.
Input Validation	Pydantic models validate request bodies on all POST	Implemented	Add rate limiting middleware to FastAPI to

Security Concern	Current Implementation	Status	Recommendation
	endpoints; validate_policy() checks policy names		prevent abuse of LLM-backed endpoints.
Dependency Security	Requirements pinned in requirements.txt; includes crewai[google-genai]	Implemented	Integrate automated vulnerability scanning (e.g., Trivy) in the CI/CD pipeline for Docker images.
Log Sanitisation	LLM prompts and API keys are not explicitly logged	Implemented	Implement structured logging with explicit exclusion of sensitive fields.

Security Risk Summary:

Risk	Severity	Mitigation
Kubernetes Secrets only base64-encoded (not encrypted at rest)	Medium	Enable etcd encryption at rest in Minikube / production cluster configuration.
Containers run as root user	Medium	Add non-root USER directive to Dockerfile. Use read-only filesystem where possible.
No Network Policies isolating pods	Medium	Define Kubernetes NetworkPolicy resources to whitelist only necessary pod-to-pod traffic.
No HTTPS on internal service traffic	Low (internal only)	Acceptable for Minikube local development. Use TLS in production via Ingress.
No rate limiting on LLM endpoints	Low-Medium	Add slowapi or similar FastAPI rate limiting middleware to prevent excessive Gemini API calls.

7.2 Performance Aspects

Performance characteristics are primarily governed by the latency of external LLM inference calls. The system is designed to handle these gracefully with appropriate timeouts and a multi-replica backend deployment.

Performance Metric	Current Target / Actual	Notes
Research Request Latency	30 – 90 seconds (end-to-end)	Dominated by Gemini LLM inference for 2 sequential agent calls. Streamlit timeout set to 120s.
Comparison Request Latency	60 – 150 seconds (end-to-end)	Three sequential LLM calls (2x Researcher + 1x Comparator). Streamlit timeout set to 180s.
Health Check Latency	< 100ms	Hardcoded JSON response — no agent or LLM involvement. Fastest endpoint.
Policy List Latency	< 50ms	In-memory static list lookup. Negligible compute time.
Backend Throughput	2 concurrent requests (2 replicas)	Each pod handles one LLM crew at a time. 2 replicas support 2 simultaneous research/compare requests.
Pod Startup Time	10 – 15 seconds (readinessProbe initialDelay: 10s)	Python dependency loading + uvicorn startup. readinessProbe ensures traffic only reaches ready pods.
Rolling Update Duration	~60 – 90 seconds per replica	New pod must pass readinessProbe (10s delay + 5s periods) before old pod is removed. Zero requests dropped.
Memory Usage	256Mi request / 512Mi limit per backend pod	Accommodates Python runtime, CrewAI, and in-flight LLM response objects. Limit prevents OOM from runaway agents.
CPU Usage	250m request / 500m limit per backend pod	LLM calls are I/O-bound (HTTPS wait). CPU usage is low during inference; peaks briefly on JSON parsing.

Performance Improvement Recommendations:

Improvement	Expected Benefit
Redis LLM response cache	Eliminates repeat Gemini calls for previously researched policies. Response time drops from ~60s to <100ms for cached queries.
Horizontal Pod Autoscaler (HPA)	Automatically scales backend replicas based on CPU/memory usage, increasing throughput during peak demand.
Async FastAPI endpoints	Running agent crews in async tasks (FastAPI BackgroundTasks or asyncio) would allow the backend to accept new requests while LLM inference is in progress.
Ingress + Load Balancer	Replace NodePort + SSH tunnel with NGINX Ingress for production-grade request routing, connection pooling, and TLS offloading.
LLM Streaming Responses	Use Gemini streaming API to begin rendering results in Streamlit as tokens arrive, reducing perceived latency significantly.

8. References

The following technologies, frameworks, tools, and documentation sources were referenced in the design and implementation of the Multi-Agent Insurance System:

#	Reference	Type	Relevance
1	CrewAI Documentation	Framework Documentation	Agent and task orchestration patterns, Crew.kickoff() API, Process.sequential mode.
2	Google Gemini API (google-gemini)	External API Documentation	LLM model selection (gemini-2.5-flash), API key authentication, prompt/response format.
3	FastAPI Official Documentation	Framework Documentation	REST endpoint definition, Pydantic validation models, JSONResponse, uvicorn server configuration.

#	Reference	Type	Relevance
4	Kubernetes Official Documentation	Platform Documentation	Deployment, Service, ConfigMap, Secret, RollingUpdate strategy, readinessProbe, livenessProbe specifications.
5	Minikube Documentation	Tool Documentation	Local Kubernetes cluster setup on Windows, Docker driver, minikube service tunnel, docker-env configuration.
6	Docker Official Documentation	Tool Documentation	Dockerfile best practices, multi-stage builds, layer caching, imagePullPolicy: Never configuration.
7	Streamlit Documentation	Framework Documentation	st.session_state, page layout, timeout handling, environment variable access for BACKEND_URL.
8	Python Requests Library	Library Documentation	HTTP timeout configuration (timeout=120s / 180s), exception handling for ConnectionError and Timeout.
9	LiteLLM Documentation	Library Documentation	LLM abstraction layer used by CrewAI to interface with Gemini; handles API key injection and response parsing.
10	Kubernetes NetworkPolicy	Security Reference	Pod-to-pod traffic restriction patterns referenced for future security hardening recommendations.
11	NGINX Ingress Controller	Infrastructure Reference	TLS termination and path-based routing patterns referenced for production deployment recommendations.

#	Reference	Type	Relevance
12	Pydantic V2 Documentation	Library Documentation	Request body validation model definitions used in FastAPI endpoints.